C12W7 · TUESDAY, MARCH 3, 2026

# VIBE CODING IS DEAD

Long Live *Agentic Engineering*

# PART 1: VIBE CODING IS DEAD

# IT WAS THE BEST OF VIBES

Andrej Karpathy, **Feb 2025**:

*"I 'Accept All' always, I don't read the diffs anymore."*

Andrej Karpathy, **Feb 2026**:

*"Vibe coding is passe. I'd like to mass-propose agentic engineering."*

👤 **Andrej Karpathy** — founding member of OpenAI, former Sr. Director of AI at Tesla, Stanford PhD

# HOW WE GOT HERE

**0**

### MANUAL

No AI assistance

**1**

### AUTOCOMPLETE

Tab to accept suggestions · Copilot, 2022

**2**

### CHAT-ASSISTED

Ask questions, copy answers · ChatGPT, 2023

**3**

### VIBE CODING

Accept All, hope it works · Karpathy, Feb 2025

**4**

### AGENTIC ENGINEERING

Orchestrate agents with human oversight · Karpathy, Feb 2026

# WHAT THE RESEARCH ACTUALLY SAYS

# THE HEADLINES

## THE GOOD

- **26%** more tasks completed · MIT/Microsoft
- **55%** faster task completion · Peng et al.
- **21%** faster at Google · Internal study

## THE BAD

- **19% slower** for experienced devs · METR
- **1.7x** more issues in AI code · CodeRabbit
- **45%** of AI code has security flaws · Veracode

⚠️ These are real studies. But every one of them has problems.

# THE FINE PRINT

### 26% MORE TASKS

Code completion, not agentic. Only counts *compliers*. Proxy metrics (PRs, commits), not value delivered. Build success **declined 5.5 pp**.

### 55% FASTER

**One JS task**, single session. Upwork freelancers. CI: [21%, 89%]. No quality measurement.

### 21% AT GOOGLE

p = **0.086**. Convention is $p < 0.05$. Not statistically significant after adjustment. Google engineers on Google tasks.

### 19% SLOWER

N=16, experienced devs on complex codebases only. Follow-up (N=57) had *worse* methodology problems — METR published their own critique.

### 1.7X MORE ISSUES

Own product measured the issues. "AI" PRs identified by commit messages — self-selected. Confounders could explain everything.

### 45% SECURITY FLAWS

Raw model output — no specs, no review, no process. 100+ models on 80 isolated tasks. Worst case only.

Every study has problems. The question is *which direction* those problems push the results.

# THE VENDOR CONFLICT PROBLEM

Of the **22 major studies** on AI-assisted coding:

| 7 | 5 | 4 |
|---|---|---|
| FROM AI TOOL VENDORS | FROM COMPETITORS | CLEAN ACADEMIC PROFILES |

- **CodeRabbit** found "1.7x more issues" in AI code — using their own product as the measurement tool. They sell AI code review for $24/dev/month.
- **Veracode** found "45% security flaws" — no specs, no review, no process at all. They sell security scanning tools.

# WHAT WE CAN CONCLUDE

1. **AI increases throughput** with varying quality
2. **Juniors may benefit more** than seniors
3. We might **overestimate AI's benefit**
4. **At big orgs, complex existing codebases, gains are modest**
5. **But at startups, AI is becoming the primary coder**
6. **The field is moving faster than the research.** Cherny stopped writing code by hand in Nov 2025 — 4 months ago. No study has measured mature agentic workflows.
7. **Long-term velocity is unmeasured.** AI-assisted refactoring, codebase comprehension, and onboarding should improve velocity, but we need longitudinal studies to see it.
8. **AI is an amplifier**. good fundamentals get better results

**AGENTIC ENGINEERING IS A SKILL TO BE LEARNED,**

**NOT A MAGIC WAND.**

# THREE RULES FOR THE BUILD PHASE

# RULE 1: THE WILLISON RULE

> *"If an LLM wrote the code for you, and you reviewed it, tested it, and can explain how it works to someone else — that's not vibe coding, it's software development."*
>
> — Simon Willison

⚠️ **Class rule:** Don't deploy code you can't understand.

# RULE 2: SPEC FIRST, CODE SECOND

Every credible high-output practitioner uses **spec-first development:**

### BORIS CHERNY

Creator of Claude Code, Anthropic

Plan Mode → iterate until solid → one-shot it

### HARPER REED

Former CTO, Obama 2012 campaign

spec.md → blueprint → execute

### ADDY OSMANI

Engineering leader, Google Chrome

"Waterfall in 15 min" — spec before code

### KENT BECK

Creator of TDD & Extreme Programming

TDD cycle: Red → Green → Refactor

You already have `/feature-forge`: **brief → architecture → plan → implement → review**

# RULE 3: CLAUDE.MD IS YOUR TEAM'S BRAIN

## WHAT IT IS

- Persistent context file in your repo root
- Claude reads it at the start of **every session**
- Your project's identity: stack, conventions, structure
- 67% of repos now use them

## HOW TO USE IT

- Root CLAUDE.md: Root: under 60 lines. Project: up to ~40k tokens
- Check it into git — compounds in value
- When Claude does something wrong, **add a note**
- Template on the one-sheet

*"Anytime we see Claude do something incorrectly, we add it to the CLAUDE.md."*

— Boris Cherny

# GIT FOR TEAMS

With **2-4 people coding**, this prevents chaos:

- **Never push to main directly.** Feature branches + PRs.
- **One feature per branch.** One AI session per branch.
- **Pull before you branch.** Commit often. Small PRs.
- **Someone reviews every PR.** Read the code, don't just check that tests pass.

⚠️ If two of you push to main at the same time without branches, **you will enter merge hell.**

# PART 2: MVP SCOPING

# 13 TEAMS, 10 DAYS

| TEAM | PRODUCT | BUILDING |
|---|---|---|
| **4CUz** | NoticeMe | Social accountability for screen time |
| **August Fan Club** | Evidence of Life | Auto-timeline from your Apple Calendar |
| **Basket Architects** | Experience Architects | Living taste profile from real-world experiences |
| **Dream Team** | Campus Study Match | See who nearby is studying the same subject |
| **J-Combinator** | — | Curated NYC event recommendations |
| **LLMao** | LLMao | Activity-based social for new social circles |
| **Millionaires** | QuestCity | Gamified real-world exploration with quests |
| **misfitAI** | Wardrobe Planner | Outfit suggestions from calendar + weather + wardrobe |
| **Roam** | Roam | Matching people on NYC activity routes |
| **Standard Deviants** | — | Qualitative/mixed-methods data analysis tool |
| **team-friendly** | — | TBD |
| **team-frontend** | FreeWeight | Strength training programming + tracking |
| **The Birds** | Circle | One-tap social coordination; see live hangouts |

# THE 10-DAY SPRINT

| Today | Thu | Mar 10 | Mar 12 |
|---|---|---|---|
| DEFINE + SET UP | BUILD SPRINT | BUILD SPRINT | MVP PRESENTATIONS |

Everything before this was validation. **Now you ship.**

# WHAT IS AN MVP?

> *"The minimum viable product is that version of a new product which allows a team to collect the maximum amount of validated learning about customers with the least effort."*
>
> — Eric Ries, The Lean Startup

- **Minimum** = least effort. **Viable** = delivers real value. Both matter.
- Not a prototype (tests usability) — an MVP **tests the business hypothesis**
- YC's Michael Seibel: "Hold the problem tightly, hold the solution loosely"
- Reid Hoffman: "If you aren't embarrassed by the first version, you shipped too late"

# YOUR MVP = FIRST AHA MOMENT

Your MVP = the shortest path to your user's **first aha moment**.

### YOUR MVP

- The **ONE** core flow
- User arrives → does the thing → gets value
- Demo-able in 3 minutes

### NOT YOUR MVP

- "All the features we validated"
- "Everything on our landing page"
- Settings, profiles, admin panels

💡 If you can't demo it in 5 minutes, you've scoped too much.

# WHAT CAN YOU FAKE?

### MARKETPLACES

Manually match supply and demand behind the scenes

### ALGORITHMS

Start with the most naive version, or do it by hand.

### DATA-HEAVY

Use sample or mock data

### SOCIAL / MATCHING

Seed with your team, friends, classmates

"Any sufficiently advanced demo is indistinguishable from a product."

# PART 3: LET'S BUILD

# WORK BLOCK: TWO TRACKS

### TRACK A: BUILD (MOST OF TEAM)

1. **Define your MVP scope** (10 min)
   2. What's the core flow?
   3. What are you cutting?
   4. What's faked?
   5. Who's building what?
6. **Set up dev environment**
   7. Create CLAUDE.md (template on one-sheet)
   8. Set up git branching
   9. Start first feature stub

### TRACK B: DEMAND GEN (1-2 PEOPLE)

- Landing page traffic **doesn't stop**
- Check analytics: visits, signups, drop-off
- Iterate on copy if conversion is low
- Keep running ads, social, outreach

💡 Don't stop driving traffic just because you're building.

# WRAP UP

# ASSIGNMENT: MVP DOC

DUE THURSDAY

Submit to your team folder as `20260305/mvp_doc.md`:

- **Core flow** — user → action → value
- **Tech stack** — what you're building with
- **Team roles** — who's building what, who's on demand gen
- **What's faked** — WoZ / concierge strategies
- **Demand gen status** — what's running, what are the numbers

# WHAT'S NEXT

- **Thursday (c13w7)** — Build sprint. MVP docs due. Come ready to build.
- **Tuesday Mar 10 (c14w8)** — Build sprint.
- **Thursday Mar 12 (c15w8)** — **MVP Presentations.** You demo what you built.
- **Supplemental** — Dev Workflow + Git podcast (releasing this week)

You have 10 days. The workflow is set. The one-sheet is your reference.

**Go build.**

# END OF CLASS 12

Next: Build Sprint (Thursday)